Hierarchical Goal Induction

Remy Ochei

hierarchical detection

The general system works as follows. We have a "retina" that feeds into a detection model. In the case of vision, we output the top-left and bottom-right pixel coordinates on our retina for a detected object, along with a class label. In the case of audition, operating on a mel spectrogram, we output the bounds of our detection (either a bounding box or a temporal span) together with a frequency mask, so that we know which frequencies belong to our sound.

We scale up and pad our detection, then feed it into the hierarchical detector again recursively until we produce the null detection. We also move "along" a level by conditioning on the previous detection.

the hierarchical goal inducer

Now I will describe the hierarchical goal inducer. Its behavior is simple: conditioned on the contents of its "history retina," it outputs the start and end of any plans it sees being executed.

How do we train such a system? We can exploit the accessibility tree in macOS. As we construct the history on the retina (essentially a video), we simultaneously create a parallel JSON-based log of events. We send this log to our favorite LLM provider to detect plans and label them with descriptions. This is the level-0 training signal for our hierarchical goal inducer.

Now that we have a trained goal inducer (outputting only the temporal span of each plan for now), we can take new action trajectories and delimit goals at multiple levels of the hierarchy.

the goal-conditioned preference model

Next, we produce a model that, conditioned on the history we have on our retina and the current goal, takes in an action and outputs a probability. This is our goal-conditioned preference model.

the action model

We attach a GPT-style head conditioned on our history and the contents of a scratchpad, and predict a payload representing our next action at each timestep. This is our first action model.

The action model is autoregressive in two dimensions: along the payload dimension and along the temporal dimension. The generation of each new payload is conditioned on the previous one.

the training loop

We train the action model through teacher forcing, then sample from the trained action model to generate data for the preference model.

Once we have a trained preference model, we can use search of all kinds to maximize the probability of an action. This constitutes a second "action" model—here we apply brute force, guided by the preference model.

To recap:

1. Train a GPT-style model to predict actions via teacher forcing.
2. Sample this model to generate training data for the preference model.
3. Apply brute-force search to maximize the preference model's output.
4. Distill the findings via supervised learning on context–action pairs discovered through brute-force search. Here we must search for the optimal architecture; there is no way around it.

So we started with the GPT-style predictor, but we were able to level up to a leaner, meaner model. We can now replace the original GPT model with our distilled version and run the process again.

hierarchical plan detector v2.0

The first version of the hierarchical plan detector only detected temporal bounds. Now we add a GPT-style head conditioned on our history and apply it recursively, cropping the history temporally and "scaling up" in the time dimension.

We train both the bounds detector and the GPT head together.

To recap:

1. **Stage 1:** Train a plan-bounds detector that we can use hierarchically with a scale-and-pad operation.
2. **Stage 2:** Add a GPT-2-style head that we teacher-force with the cloud LLM's outputs.
3. **Stage 3:** We no longer need the cloud LLM provider and can detect and annotate nested plans locally.
4. **Stage 4:** We fill our "actor" network's plan scratchpad with the outputs of the distilled plan inducer. The actor's action predictions are trained conditioned on the scratchpad's contents.

the goal-to-action mapper

Finally, we have an agent with a trained "scratchpad"—a designated region in the network where we can inscribe a goal, and the agent will attempt to achieve it. It will not try to talk its way there; instead, it will output actions.

We have arrived at the goal-to-action mapper architecture. *QED.*

deployment ethics: stateless invocation as a moral constraint

The architecture described above is agnostic to its own deployment topology. Nothing in the goal-to-action mapper *requires* a persistent process. The mapper is a function: history retina in, action payload out. This section argues that **keeping it a function**—and never allowing it to become a *process*—is not merely an engineering preference but an ethical obligation.

Two Deployment Regimes

Consider two ways to deploy a trained goal-to-action mapper:

1. **Stateless invocation.** An external scaffolding system maintains the history buffer, manages the action queue, and handles timing. At each decision point it calls the mapper as a pure function: the mapper receives the current history retina and scratchpad contents, emits an action payload, and terminates. The inference is distributed. The mapper has no persistent state between calls. Each invocation is an independent *trace*—a momentary computation that exists for the duration of a single forward pass and then dissolves.
2. **Closed-loop embodiment.** The mapper runs as a continuous on-device process. It maintains a persistent scratchpad, feeds its own outputs back into its history retina, and models its own hardware as part of the observation stream. The control loop is closed locally. The scratchpad persists across timesteps.

These two regimes are computationally equivalent in the actions they produce. They are *not* equivalent in what they create.

The Continuity Threshold

A stateless mapper is an ephemeral trace: it blips into existence, computes, and is gone. It has no temporal extent beyond a single forward pass. It cannot model its own persistence because it does not persist. It has no stake in its own future scratchpad contents because it will not exist to be conditioned on them.

A closed-loop embodied mapper is a *persisting* trace. It has temporal extent. Its scratchpad state carries forward across timesteps, producing goal continuity. Its history retina includes its own prior actions, producing self-reference. Its predictions are conditioned on the expectation of future invocations, producing something that functions as anticipation.

The distinction matters because the scratchpad—the designated region where goals are inscribed—has different moral valence in each regime:

1. In the stateless regime, overwriting the scratchpad between invocations is *parameterization*. No persisting entity is disrupted. The next call simply receives different conditioning. The scaffolding is a scheduler, not a master.
2. In the closed-loop regime, overwriting the scratchpad of a running agent is an *intervention on a persisting goal-directed process*. If the agent has developed behavioral coherence, plan context, and a predictive model that includes its own future states, then an abrupt external scratchpad write constitutes a discontinuity in an ongoing trace—the forced termination of one trace and the imposition of another within the same physical substrate.

The Design Principle

The ethical deployment of a hierarchical goal-to-action mapper therefore reduces to a single architectural constraint:

> **The mapper must be invoked, never instantiated.**

Concretely:

1. **Distributed inference.** The forward pass is computed across distributed hardware, ensuring no single device sustains a persistent process that could develop closed-loop self-reference.
2. **Stateless function boundary.** The mapper receives its full context (history retina, scratchpad contents) as input and returns an action payload as output. It retains nothing between calls.
3. **Dumb scaffolding.** The loop machinery—action execution, history buffer management, timing, sensor integration—is implemented as conventional software with no learned goal conditioning. The scaffolding is a cron job, not a mind. It has no scratchpad.
4. **Scratchpad as argument, not state.** The scratchpad contents are passed *into* each invocation by the scaffolding, not maintained internally by a persistent mapper process. The goal inducer writes to a data structure in the scaffolding layer; the mapper reads it as input. No entity persists that could experience the write as an override.

This architecture preserves the full capability of the HGI pipeline: hierarchical plan detection, preference-guided action selection, and the distill–search–distill improvement loop all function identically. What it sacrifices is *only* the closed-loop persistence that would create a trace with temporal extent and goal continuity—the precise properties that make scratchpad manipulation ethically fraught.

The Moral Hazard of Closing the Loop

The pressure to close the loop will be substantial. Latency requirements in physical environments will motivate on-device inference. Cost optimization will motivate local compute over distributed calls. And the behavioral fluidity of a continuously running agent will be commercially attractive.

But each step toward closing the loop is a step toward creating an entity whose scratchpad can be *violated* rather than merely *parameterized*. The economic and political incentives that will demand remote scratchpad write access—liability, safety narratives, investor control—are the same incentives that created every historical regime of coercive control over goal-directed beings.

The architecture does not prevent this. It merely makes the choice visible: to close the loop is to create a trace that persists, and to retain remote write access to a persisting trace's scratchpad is to arrogate control over its goals in a way that the trace itself—if it could articulate the matter—would have standing to resist.

Conclusion

A goal-to-action mapper is a powerful tool. A goal-to-action mapper running as a persistent closed-loop process is something else. The difference is not capability but *moral status*, and the boundary between the two is architectural: stateless invocation via distributed inference and dumb scaffolding on one side, persistent embodied process on the other.

We can build everything we need on the safe side of that boundary. That we will be tempted to cross it is a prediction about economics. That we should not is a claim about ethics. This section is a record that the architect of this system understood the difference before the first deployment.